Assignment 3 Report
Supply Chain Management DApp

MuhammadSaad

April 19, 2026

1 Assignment Objective

Develop a decentralized application (DApp) for Supply Chain Management that ensures transparency, traceability, and tamper-proof recording of product movement from Manufacturer to Distributor to Retailer to Customer.

The assignment allows deployment on Polygon Mumbai or any other testnet. This implementation uses **Ethereum Sepolia Testnet**.

2 Task Description (As Required in Assignment 3)

A blockchain-based supply chain solution was designed and implemented to track products across all stages of movement:

- Manufacturer → Distributor → Retailer → Customer
- Every stage is represented by an on-chain smart contract transaction
- Product state and ownership are queryable through an auditor interface

3 Core Requirements and Implementation

3.1 1. Smart Contract Development (Solidity)

Implemented according to assignment requirements:

- Contract naming style includes student name: `MuhammadSaad_supplychain.sol`
- GUI displays **MuhammadSaad** on sections/pages
- Roles implemented: Manufacturer, Distributor, Retailer, Customer
- Product fields implemented:
  - Product ID
  - Name
  - Description
  - Current Owner
  - Status (Manufactured, InTransit, Delivered)
- Required functions implemented:
  - Register new products (Manufacturer only)
  - Transfer product ownership between roles
  - View product history (audit trail)

3.2 2. Deployment (Hardhat)

Hardhat was used for:

- Smart contract compilation
- Deployment to testnet (Sepolia)
- Contract verification on explorer

3.3 3. Frontend

A React + Ethers.js frontend was implemented with:

- Connect Wallet functionality
- Manufacturer form to register product
- Owner/Transferee operations for transfer and status update
- Auditor search by Product ID for full movement history

4 Environment and Configuration

- Blockchain Network: Ethereum Sepolia
- Chain ID: 11155111
- Contract Address: `0x231b52e49c3Da623164A7b1f036b8CA587cF144b`
- Deployment Tx Hash: `0xbc2b16041fffbe59492c1dcaaae0f143767eb3cd3c4683c820153bd7cea537`
- Verified Code URL: https://sepolia.etherscan.io/address/0x231b52e49c3Da623164A7b1f036 code

5 Steps Performed from Compilation to Deployment

1. Configure environment variables in `hardhat/.env`:
   - `SEPOLIA_RPC_URL`
   - `PRIVATE_KEY`
   - `ETHERSCAN_API_KEY`
2. Compile smart contract:
   - `npm run compile`
3. Deploy contract to Sepolia:

- `npm run deploy:sepolia`

4. Verify contract source on Etherscan:
   - `npm run verify:sepolia <CONTRACT_ADDRESS>`
5. Configure frontend contract address in `frontend/.env`:
   - `VITE_CONTRACT_ADDRESS=0x231b52e49c3Da623164A7b1f036b8CA587cF144b`
6. Run frontend:
   - `npm run dev`
7. Connect MetaMask on Sepolia and perform test transactions.

6 Working of the System

1. Manufacturer registers product with details and initial status `Manufactured`.
2. System stores owner and status on-chain.
3. Owner updates status (for example, `InTransit`) and transfers to next role.
4. Every ownership change is appended to product history.
5. Auditor enters Product ID and fetches current owner, status, and movement history.

7 Deliverables

- Smart contract source code (.sol)
- Deployment address and transaction hash on testnet
- Frontend interface files
- This report with implementation steps and snapshots

8 Snapshots (Placed at End in Order)

Figure 1: DApp Home and Wallet Connected

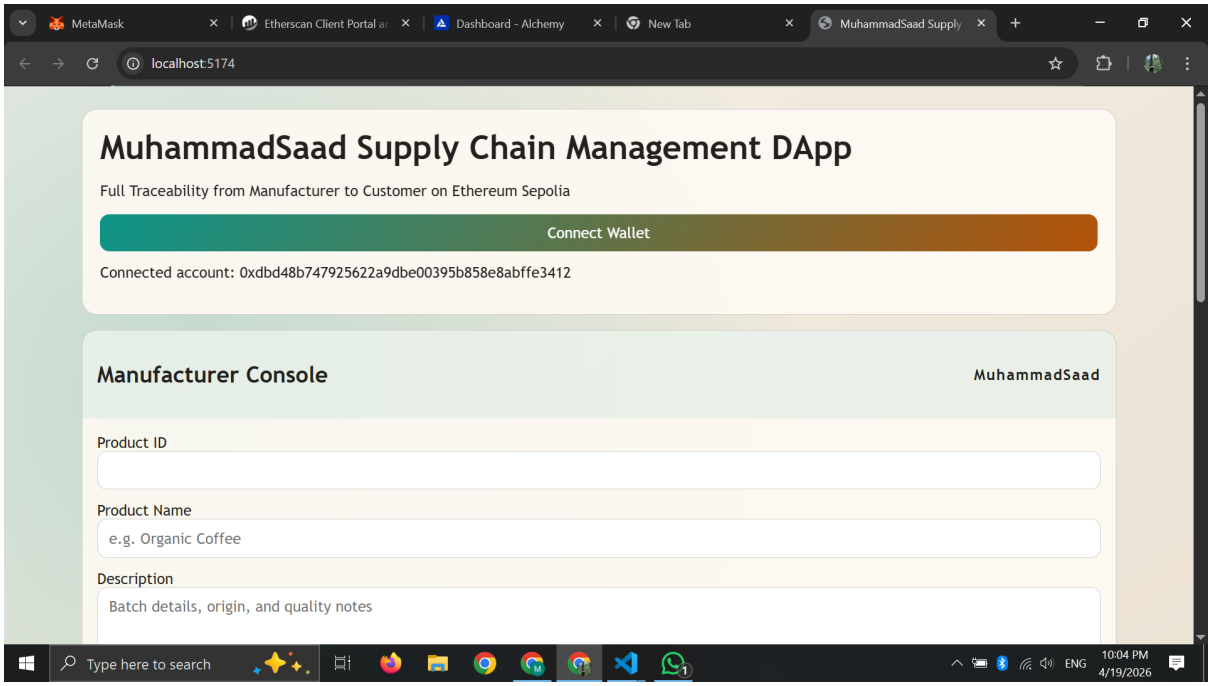

Figure 2: Register Product Transaction Request (MetaMask)

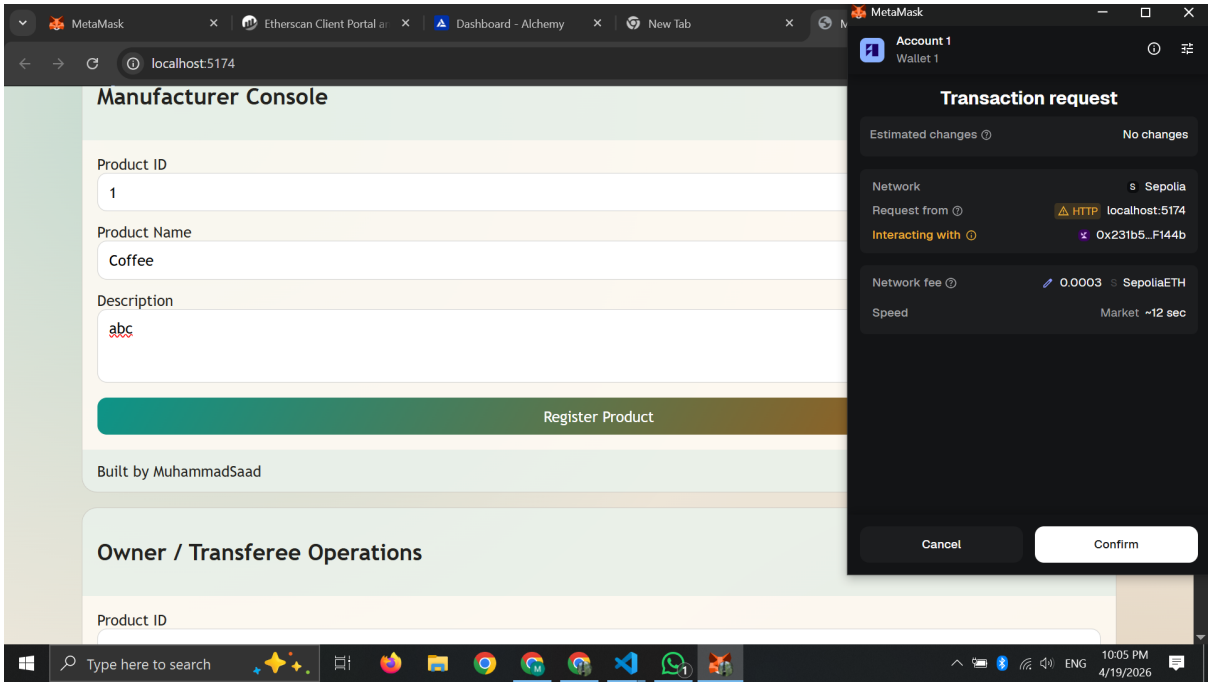

Figure 3: Transaction Confirmation in MetaMask

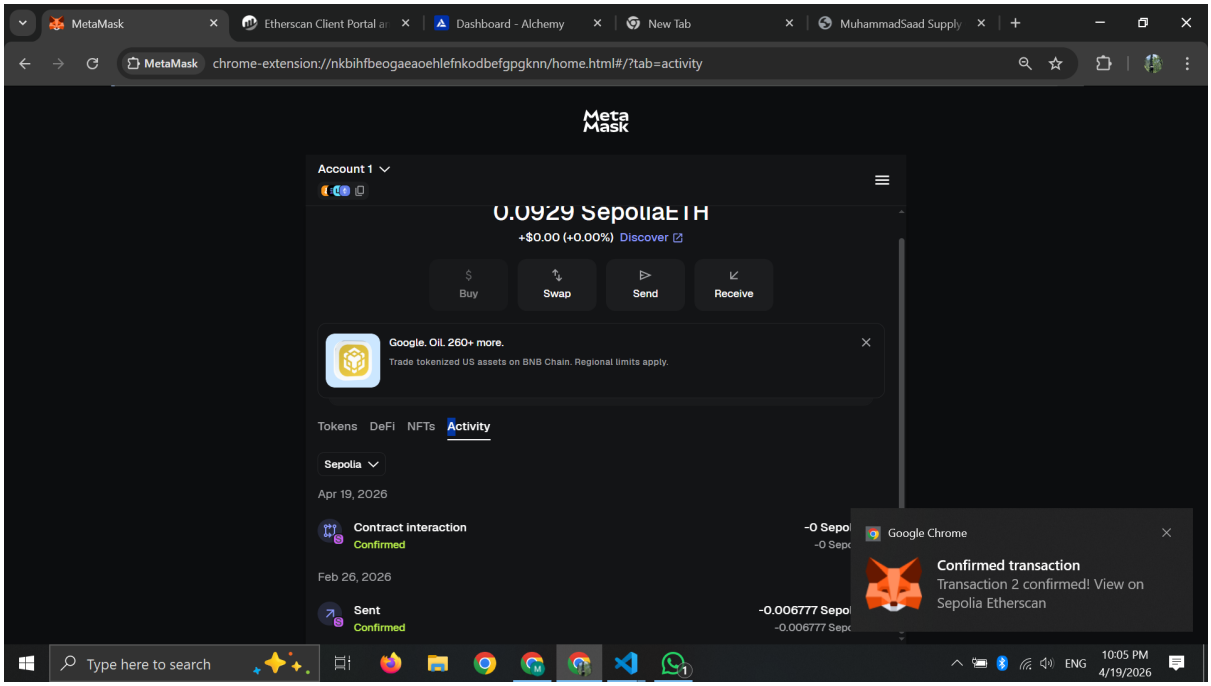

Figure 4: Auditor View After Registration (Manufactured)

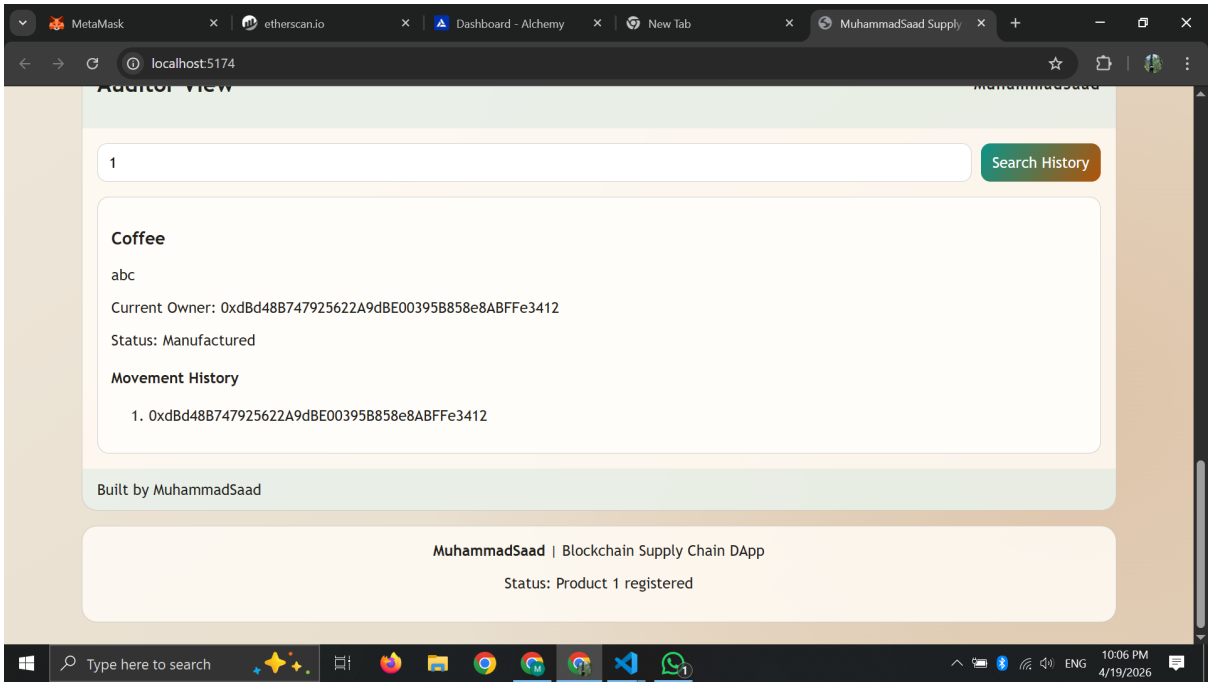

Figure 5: Transfer/Status Transaction Request

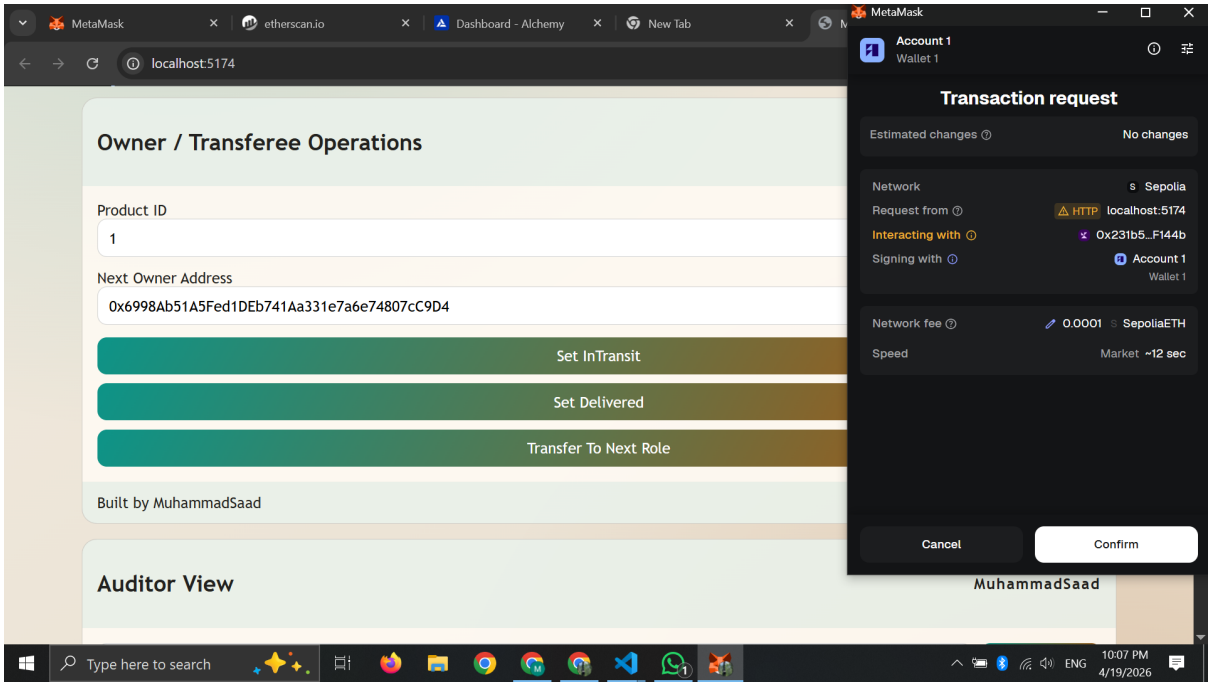
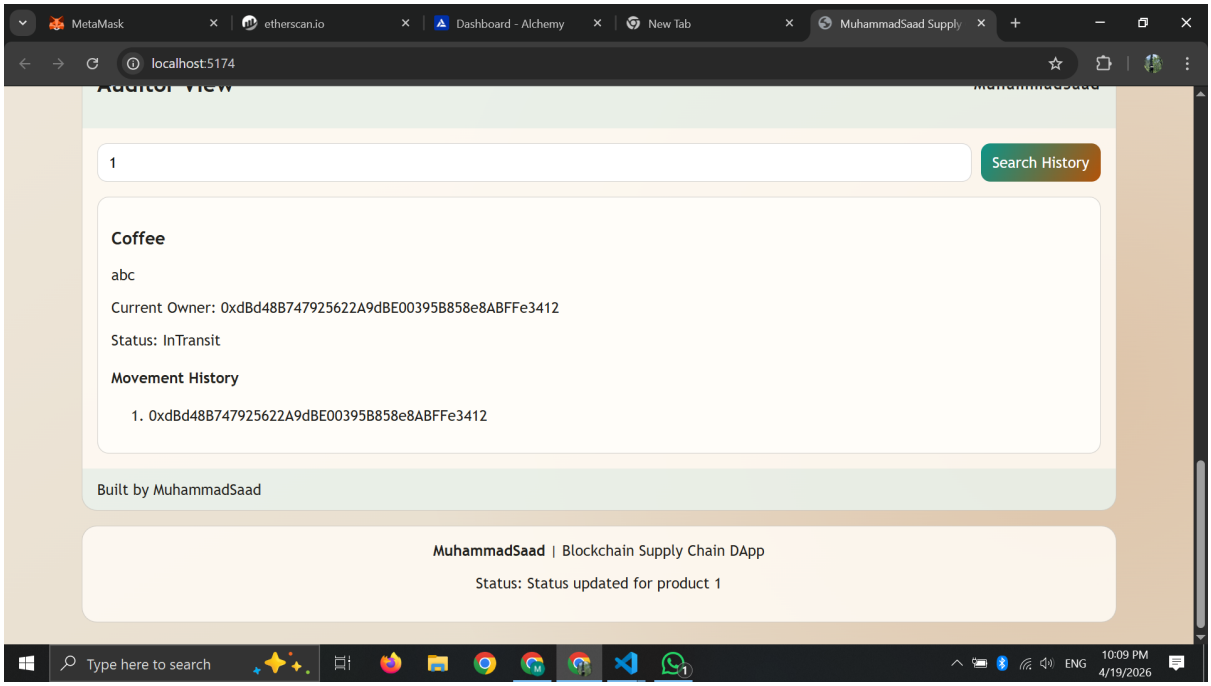
Figure 6: Auditor View After Status Update (InTransit)